

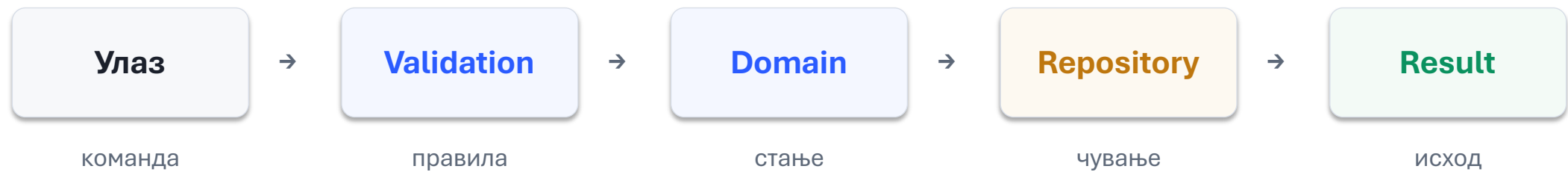


# Пословна логика и use-case слој

Границе пословних правила и организација use-case сценарија

## Use-case повезује захтев са пословним исходом

Он оркестрира доменске објекте и уговоре, али не припада UI-y, бази или transport-y.



### КЉУЧНА ПОРУКА

Use-case треба да има јасан улаз, експлицитан резултат и контролисане зависности.

## Entitet није само DTO

Јавни сетери и правила у сервисима лако воде у неважеће стање.

**Доменски објекат треба да штити своје инваријанте и да излаже операције које имају пословно значење.**

---

### КЉУЧНА ПОРУКА

**Use-case координише, али не мора да понавља исто правило при сваком позиву.**

# Factory метод чува валидно стање

Очекиван пословни неуспех враћа Result, не генерички exception.

```
1 public sealed class ReservationMaker
2 {
3
4     public Result<Reservation> Create(DateTime from, DateTime to)
5     {
6         if (from >= to)
7             return Result<Reservation>.Fail("InvalidPeriod");
8         return Result<Reservation>.Ok(new Reservation(from, to));
9     }
10 }
```

## Шта је важно

Конструктор може бити приватан.

Factory спречава очигледно невалидно стање.

Метод ентитета не треба да имају пословну логику – она остаје у сервисима.

# Инваријанта је правило које увек важи

Она се не сме ослањати на добру вољу позиваоца.

01

## Период

почетак мора бити пре краја

02

## Стање

отказана резервација се не  
отказује поново

03

## Време

прошла резервација се не  
мења као будућа

## Очекивани неуспех није грешка система

Заузет термин није пад апликације, већ исправно одбијен захтев.

**Систем ради исправно када стабилно врати `OverlappingReservation` и не промени стање које не сме да се промени.**

---

### КЉУЧНА ПОРУКА

Овакво понашање мора бити експлицитно и тестабилно.

# Result моделује пословни исход

null, false и генерички exception сакривају разлог неуспеха.

```
1 public sealed record Result(bool Success, string? Error)
2 {
3     public static Result Ok() => new(true, null);
4     public static Result Fail(string error) => new(false,
5         error);
6 }
7
8 public sealed record Result<T>(bool Success, T? Value,
9     string? Error);
```

## Шта је важно

Код неуспеха је стабилан.

Клијент не мора да тумачи изузетак.

Тест може директно да провери очекиван исход.

## Различите ситуације траже различит модел

Не треба све третирати као exception.

| Ситуација                      | Препоручени модел                     |
|--------------------------------|---------------------------------------|
| Неважећи пословни захтев       | Result и стабилан код неуспеха        |
| Ентитет није пронађен          | NotFound исход или Result код         |
| База недоступна                | инфраструктурна грешка                |
| Нарушена доменска претпоставка | експлицитан неуспех у пословном слоју |

# Use-case сервис оркестрира операцију

Он користи Domain и уговоре, али не преузима све одговорности.

```
1 public sealed class ReservationService
2 {
3     private readonly IReservationRepository _repository;
4     private readonly IClock _clock;
5
6     public Result Reserve(ReservationRequest request)
7     {
8         if (request.From <= _clock.UtcNow)
9             return Result.Fail("ReservationMustBeInFuture");
10        if (_repository.HasOverlap(request.EquipmentId,
11            request.From, request.To))
12            return Result.Fail("OverlappingReservation");
13        return Result.Ok();
14    }
15 }
```

## Шта је важно

Use-case проверава ток операције.

Repository је уговор, не конкретна база.

Време је зависност која се контролише.

# Dependency Injection није само framework

То је начин да се променљиве зависности добију споља.

01

## IClock

контролише време

02

## Repository

складиште као уговор

03

## Service

оркестрација use-case-a

04

## Composition root

бира конкретне имплементације

## Пословна правила не припадају controller-у

UI/API треба да преведе улаз, позове use-case и мапира резултат.

### CONTROLLER

прима захтев

валидација формата

позива application use-case

мапира Result у HTTP или UI одговор

### USE-CASE

проверава пословни ток

позива доменске операције

користи портове

враћа експлицитан исход

## Use-case треба да буде кориснички смислена операција

То није свака метода у коду.

**Добар use-case може да се објасни од захтева до резултата: шта корисник покушава, која правила важе, које стање се мења и који исход систем враћа.**

---

### КЉУЧНА ПОРУКА

То је јединица пројектовања и одбране.

## Где се смешта правило

Положај правила зависи од тога шта оно зна и шта штити.

01

### Entity

инваријанте сопственог стања

02

### Domain service

правило више доменских објеката

03

### Application service

оркестрација use-case-a

04

### Infrastructure

технички адаптер без пословне одлуке

## Стабилни кодови исхода су део уговора

Клијент треба да зна шта значи неуспех.

| Код                       | Значење                   |
|---------------------------|---------------------------|
| InvalidPeriod             | почетак није пре краја    |
| ReservationMustBeInFuture | термин није у будућности  |
| OverlappingReservation    | период се преклапа        |
| AlreadyCancelled          | операција је већ извршена |

## P2 фокус

Функционално језгро мора бити објашњиво.

2

### кључна use-case-a

До контролне тачке треба имати најмање два use-case-a са експлицитним улазима, резултатима и јасним границама одговорности.

---

## Пројектна контролна тачка Р2

Важно је централно понашање, не завршени екрани.

- 01      Имплементирати најмање два кључна use-case-а.
- 02      Пословна правила не расипати по controller-у, менију и repository-ју.
- 03      За очекиване неуспехе дефинисати стабилне кодове или типове резултата.
- 04      Спољне зависности уводити кроз уговоре и composition root.
- 05      У PR-у описати најмање једну архитектонску одлуку.

## Мини домаћи: bool/null y Result

Метода која враћа bool не објашњава зашто је операција одбијена.

```
1 // пре
2 bool Reserve(Request request);
3
4 // после
5 Result Reserve(Request request);
6
7 // тест проверава
8 Assert.That(result.Success, Is.False);
9 Assert.That(result.Error, Is.EqualTo("OverlappingReservatio
10     n"));
```

### Шта је важно

Рефакторисати једну методу са bool/null исходом.

Додати два мала теста или демонстрациона сценарија.

Образложити нови стабилан код неуспеха.

## Добар use-case има три видљиве границе

Без њих је тешко доказати исправност.

01

### Input

команда или захтев

02

### Policy

пословно понашање

03

### Output

експлицитан исход

## Суштина Вежбе 3

Пословна логика мора бити на месту где може да се разуме, тестира и брани.

**Use-case није техничка основа, већ контролисан ток који повезује захтев, доменска правила, зависности и стабилан резултат.**

---

### КЉУЧНА ПОРУКА

Тако се припрема основа за тестирање.

## За наредни термин

Припремити код који може да се тестира из пословних сценарија.

- 01      Изабрати два use-case-а у пројекту.
- 02      За сваки дефинисати улаз, пословне исходе и стабилне кодове грешке.
- 03      Проверити да ли доменски објекти штите своје инваријанте.
- 04      Изместити пословна правила из UI/API слоја.
- 05      Припремити сценарије за NUnit тестове.